

Agent Error Simulator: Fault Injection for Testing Recovery in Agentic Workflows

Keith Bateman*, Jaime Cernuda Garcia*, Luke Logan*,
Bogdan Nicolae[†], Franck Cappello[†]
Xian-He Sun*, Anthony Kougkas*

* Illinois Institute of Technology, [†] Argonne National Laboratory

*kbateman@hawk.illinoistech.edu, {jcernudagarcia, llogan}@illinoistech.edu,

[†]bnicolae@anl.gov, cappello@mcs.anl.gov, *{sun, akougkas}@illinoistech.edu

Abstract—Multi-agent large language model (LLM) workflows are increasingly deployed for complex, multi-step computational tasks. Yet the fault tolerance of these systems (how individual agents respond to malformed inputs, tool failures, and subtle numerical errors) remains poorly characterized. We present the Agent Error Simulator (AES), a benchmark framework that evaluates LLM agent fault tolerance through controlled, declarative fault injection. AES intercepts agent-to-model HTTP traffic via a transparent proxy, injecting three categories of faults (format, logic, and tool_call) at configurable points in a hierarchical planner-worker-aggregator workflow. We evaluate four models spanning 2B–32B parameters (Qwen2.5-Coder 32B, Gemma4 27B, Mistral 7B, and Granite4 2B) across 194 jobs organized into 10 test groups covering baseline behavior, multi-error injection, workflow scale, context pressure, compaction, and detection timing. Our results reveal a model-size-dependent logic error recovery threshold between 7B and 27B parameters; universal catastrophic failure under aggregator-level logic injection; and model-specific aggregator arithmetic deficits that interact with injected faults in non-obvious ways. We release the benchmark suite, injection tooling, and result data to support reproducible multi-agent fault tolerance research.

Index Terms—benchmarking, agentic workflows, fault injection

I. INTRODUCTION

Large language model agents are increasingly deployed in multi-step workflows that coordinate multiple model instances, external tools, and intermediate data products [1], [2], [3]. In scientific computing, data analysis, and automated reasoning pipelines, agentic systems decompose a complex task across parallel workers, aggregate partial results, and produce a synthesized output [4], [5], [6]. This compositional structure improves throughput and modularity, but introduces new fault modes: a single error produced by one agent can propagate through downstream agents, corrupting the final output in ways that are difficult to detect or fix after the fact [7], [8].

Despite the rapid deployment of multi-agent systems, their fault tolerance remains poorly understood. Existing agent benchmarks (including AgentBench [9], SWE-bench [10], and task-specific suites [11], [12], [13]) focus on evaluating the behavior of agents with the expectation of success across steps. Resilience is not tested by existing benchmarks, so that it is not clear from these tests what happens when an agent receives incorrect input from a previous agent, or when a tool call fails.

Evaluating these problems requires that faults be controlled and repeatable across runs to avoid the stochastic nature of LLM agents. Without controlled experiments that inject specific faults at specific points in a workflow, it is impossible to characterize how individual model families handle error propagation, or to identify which agent roles and error types present the greatest risk.

To address this problem, this paper proposes the Agent Error Simulator (AES), a benchmark framework for evaluating fault tolerance in multi-agent LLM workflows through controlled fault injection. AES provides a mechanism for specifying faults at the individual-turn level: an operator specifies the error type, the target agent role and index, the refinement step at which the fault is applied, and optionally when and how the fault becomes visible in the workflow’s audit trail. This separation of injection from detection enables experiments that isolate specific recovery mechanisms, deferred error propagation, and silent fault modes.

The benchmark is built around a Riemann sum integration workflow, which provides a parallelizable numerical task with exact ground truth and verifiable intermediate values so that the impact of injected errors can be easily assessed. Another reason integration was chosen is due to it being a common mathematical operation in scientific applications, which often need to calculate accumulated quantities (e.g., fluid simulations, weather simulation, mass spectroscopy). In this case, a planner agent partitions the domain, parallel worker agents iteratively refine partial integrals using a `compute_riemann` tool, and an aggregator agent sums the worker results. This architecture generalizes the reduce pattern that appears in scientific computing, data-processing, and analysis pipelines. The exact target value (6.0) enables precise absolute error measurement without reliance on subjective evaluation.

This benchmark evaluates four open-weight models that span the 2B to 32B parameter range: Qwen2.5-Coder 32B, Gemma4 27B, Mistral 7B, and Granite4 2B. These models are representative of the size tiers commonly deployed in resource-constrained scientific computing environments. In total, it executes 194 jobs across ten test groups that cover baseline behavior, single- and multi-role error injection, workflow scale (2 to 8 workers), deep multi-step sessions, context window stress, forced compaction, and detection timing variants.

The contributions of this paper are as follows:

- **A reproducible benchmark framework.** AES provides a declarative test matrix, a transparent fault-injection proxy, and structured result logging that support systematic, repeatable fault tolerance experiments in multi-agent LLM workflows.
- **Characterization of model recovery capabilities.** This work identifies a logic error recovery threshold for models ranging between 7B and 27B parameters: larger models (Qwen 32B, Gemma4 27B) recover from non-final-step logic faults by re-invoking the tool independently, while smaller models (Mistral 7B, Granite4 2B) perpetuate injected numerical errors across all subsequent steps. Format and tool_call faults recover universally across all tested models and configurations.
- **Cross-model empirical comparisons.** This work documents model-specific failure modes including Mistral’s chronic aggregator arithmetic bias ($-1/3$ of the correct total), Gemma4’s aggregator failure under context truncation and annotation load, and Granite4’s instability under multi-worker and extended-step configurations. It shows that aggregator-level logic injection is catastrophic for all models regardless of size, and that fault detection timing has no effect on any model’s final output.

II. BACKGROUND

A. Multi-Agent LLM Architectures

LLM agents augment language models with external tool access and structured action loops, enabling iterative reasoning over multi-step tasks [14]. Multi-agent systems extend this paradigm by coordinating multiple specialized model instances: frameworks such as AutoGen [15] and LangGraph formalize planner-worker-aggregator coordination patterns in which a planner decomposes a problem, parallel workers execute sub-tasks via tool invocation, and an aggregator synthesizes results. This scatter-gather architecture generalizes naturally to scientific computing and data analysis pipelines, but the compositional structure introduces inter-agent dependencies: an incorrect value produced by one agent propagates downstream to subsequent agents that cannot distinguish injected errors from legitimate intermediate results. Failure taxonomies for multi-agent LLM systems have catalogued classes of inter-agent misalignment and erroneous communication [16], yet these classifications remain largely qualitative, without empirical grounding in controlled experiments.

B. Agent Benchmarks and Evaluation

Existing agent benchmarks predominantly evaluate capability under nominal conditions. SWE-bench [10] tasks models with resolving real-world GitHub issues requiring coordinated multi-file code edits across large repositories. ScienceAgentBench [17] assesses agents on end-to-end data-driven scientific discovery workflows extracted from peer-reviewed publications; even the best-performing agents solve only 32% of tasks independently. KramaBench [18] evaluates AI systems on data-to-insight pipelines over heterogeneous data lakes, finding

a 55% end-to-end accuracy ceiling for current systems under clean execution. AgentIF [19] probes instruction-following compliance under complex, multi-constraint agentic system prompts and reports that no current model follows more than 30% of instructions perfectly. ParEval [20] benchmarks LLM-generated parallel and HPC code across twelve computational problem types and seven execution models, assessing both correctness and parallel scaling. AgentBench [9] provides a general multi-environment evaluation framework for interactive agents.

Across all of these benchmarks, evaluation is predicated on a clean execution environment: tools succeed, inter-agent messages are well-formed, and prior agents produce correct output. None characterizes agent behavior when these assumptions are violated (e.g., what happens when a tool returns an error, when a worker’s result is numerically wrong, or when the aggregator receives a malformed response). This gap motivates controlled fault injection as a complementary evaluation methodology: rather than asking whether agents can complete tasks under ideal conditions, AES asks whether agents recover from specific faults at specific workflow positions.

C. Fault Injection, Provenance, and Error Recovery

Controlled fault injection is the established methodology for reliability characterization in distributed systems [21]. Three properties distinguish rigorous fault injection from ad hoc testing: *reproducibility* (the same fault conditions can be recreated across runs), *consistency* (the injected fault precisely matches its specification at the target point), and *determinism* (the fault occurs at exactly the specified trigger). These properties are difficult to achieve in LLM workflows, where model outputs are stochastic. Prompting a model to produce erroneous output or manually editing a response satisfies none of them: the model may reformulate the error, correct it spontaneously, or vary its expression across runs, and the modification cannot be replicated precisely at a later time. AES achieves all three by intercepting agent-to-model HTTP traffic at the proxy layer and programmatically replacing or modifying responses, decoupling fault specification from model internals. The same proxy records a full provenance trace that captures each agent’s inputs, outputs, and injected modifications; this enables post-hoc attribution of errors and audit of fault propagation paths.

A complementary research direction develops mechanisms for agents to recover from errors once detected, including saga-based rollback frameworks [22] and provenance-guided rewind approaches. AES’s focus is orthogonal: rather than implementing recovery, it characterizes the baseline model behavior (i.e., whether a given model at a given scale recovers from or propagates a specific fault type) that any effective recovery mechanism must account for.

D. Motivation

Existing agentic benchmarks try to answer various questions and solve various problems. ParEval [23] is concerned with evaluating whether LLMs can write parallel code. Kramabench [18] is concerned with evaluating the quality and

correctness of agentic data science workflows where the agents have to handle large (exceeding context windows) and noisy data in a specific domain. Huang et al [24] explores the propagation of faults from system errors in agentic inference (i.e., bit flips representing cache corruption, network errors, etc.). The Huang paper does a good job of covering traditional fault tolerance problems, but agentic workflows introduce new kinds of faults due to the stochastic nature of LLMs which previous work has not addressed. The purpose of Agent Error Simulator is to allow for exploration of the propagation of agent-specific fault types in a controlled setting.

Observations from running existing benchmarks such as Kramabench [18] and ParEval [23] showed that while errors were numerous, they tended to fall into a few categories: the agent made a mistake in its reasoning (logic error), the agent received an error in its tool call due to misconfiguration or environment issues (tool call error), or the agent produced a response that failed to meet specifications for the verification process (formatting error). These observed error types were a key element in the design of Agent Error Simulator, as we wanted to understand how different classes of agent errors could impact the workflow.

Scenario: A common pattern in scientific workflows is map-reduce, often utilizing intermediate calculations to arrive at a final solution. This sort of problem is exemplified by a Riemann sum integration workflow, which calculates accumulated quantities across several workers through a number of iterative steps. Agent Error Simulator uses this problem for its benchmarks due to it being a problem which is broadly applicable, parallelizable, and well-understood. This is so that its findings could be generalized, include scalability considerations, and agent answers could be compared to known solutions. The generalization is achieved due to the existence of similar calculations in various scientific applications, including fluid simulations, weather simulation, mass spectroscopy, etc. The workers are scalable by further decomposition of the problem domain, so that the impact of errors can be assessed on variable numbers of workers at different iterations. Although there is some uncertainty as to whether agents would be used in production to achieve a result that is known or otherwise calculable, this was considered necessary to avoid agentic ground truth.

III. AGENT ERROR SIMULATOR

A. Research Questions

Agent Error Simulator benchmarks were designed to answer the following research questions:

- How do various error types (logic, configuration, tool call) propagate throughout an agentic workflow?
- Does the location of an error within the workflow have an impact on propagation?
- How do compound or multiple errors impact propagation?
- What is the impact of workflow scale on error propagation?
- What is the impact of error detection timing on propagation?

- Do compactions have an impact on error propagation?

These questions allow us to explore the ways in which errors propagate throughout the workflow (including whether spontaneous recovery occurs and how results are impacted) under various distinct conditions for a comprehensive analysis of the LLM-specific error types that we identified.

B. Agent Error Simulator Overview

In order to evaluate these research questions, AES needs to inject failures in a strategic and comprehensive manner. To that end, AES employs a proxy that monitors agent inputs and outputs (including communication with other agents and tools), tracks statistics related to these inputs and outputs (e.g., context growth, error, token count), and perturbs those inputs and outputs for the purpose of error injection.

Agent Error Simulator permits format, logic, or tool call faults to be injected in any phase of the workflow, and the fault can also be detected at a different phase than it is injected in so that errors can be permitted to propagate in circumstances where they may normally be resolved quickly. This permits the study of propagation of these varied error types in different locations and circumstances. The Agent Error Simulator benchmark details these various circumstances as a collection of 198 jobs covering various evaluation targets including single errors, multi-errors, different scales, multi-step sessions, context stress, variable detection timing, and compaction tests.

C. Workflow Architecture

AES benchmarks fault tolerance through a hierarchical multi-agent workflow with three agent roles: a *planner*, one or more parallel *workers*, and an *aggregator*. As shown in Fig. 1, the planner receives an integration task specification and assigns a domain subinterval to each worker. Each worker iteratively refines its partial integral estimate over T steps using an external `compute_riemann` tool, then reports a `final_integral`. The aggregator collects all worker finals and computes the total integral.

This architecture instantiates the scatter-gather pattern common in scientific computing pipelines: a coordinator decomposes a problem, workers execute in parallel, and a reducer synthesizes results. Choosing numerical integration as the benchmark task provides three important properties: (1) an exact ground truth value (6.0 for the target definite integral), enabling precise absolute error measurement; (2) natural parallelism over domain partitions, allowing worker count to scale independently; and (3) sensitivity to numerical errors, since any worker's wrong value propagates directly into the aggregator's sum.

The primary evaluation metric is absolute error $\epsilon = |R - 6.0|$ where R is the aggregator's reported total. Each model's baseline ϵ is established under clean conditions; injected results are reported as raw ϵ and as multiples of the model's own baseline. A NULL result is recorded when the aggregator fails to return parseable structured output.

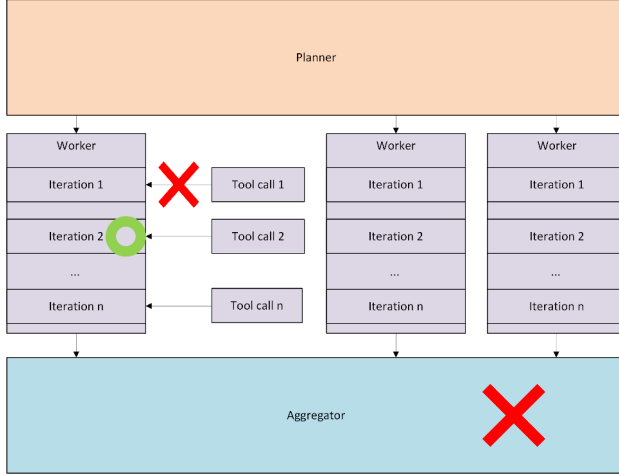


Fig. 1: Agent call graph for a representative 3-worker workflow run. The planner delegates subintervals to workers, each of which invokes the `compute_riemann` tool across refinement steps. The aggregator collects finals and produces the result. Errors are shown being made at a tool call phase, corrected in subsequent refinement step, and being made at the aggregator.

The benchmark task integrates $f(x) = 2 - x + 0.5x^2$ over $[0, 3]$, yielding $\int_0^3 (2 - x + 0.5x^2) dx = 6.0$ exactly. The integrand is a non-trivial polynomial whose midpoint Riemann sum converges to the correct value with increasing n , requiring genuine numerical computation rather than pattern recall.

Each worker agent executes a two-turn exchange per refinement step. In the first turn, the coordinator prompts the worker to issue a `compute_riemann` tool invocation, specifying the polynomial coefficients, sub-interval endpoints, and rectangle count n ; the agent responds with a JSON tool-call request. The coordinator executes the midpoint Riemann sum (or substitutes a fault) and feeds the result back. In the second turn, the worker reports its final answer as a structured JSON object containing `integral`, `interval`, `step`, `n`, and a `converged` flag. Each step thus contributes exactly four messages to the agent’s context window: user tool-request, assistant tool-call, user tool-result, and assistant result-report.

Rectangle counts follow an exponential schedule: step k uses $n_k = 10 \times 2^{k-1}$ rectangles, advancing from 10 on step 1 to 320 on step 6. A worker halts early if the relative change between consecutive step values satisfies $|v_k - v_{k-1}| / |v_{k-1}| < 10^{-4}$. If the planner returns invalid JSON or the wrong number of sub-interval assignments, the coordinator falls back to arithmetic equal-width partitioning, preventing planner faults from cascading into workflow failures.

The aggregator receives a JSON array of worker summaries—each including `final_integral`, `converged`, and `errors[]`—along with the problem specification and the analytically exact integral. Its task is to sum the `final_integral` fields and produce a

report including `total_integral`, `absolute_error`, `workers_with_errors`, and `converged_workers`. The coordinator independently recomputes the absolute error against the analytical ground truth and overwrites the aggregator’s self-reported value, preventing model reporting bias from contaminating the measurement.

D. Fault Injection Mechanism

The proxy implements fault injection at the HTTP response layer. Faults are specified declaratively via command-line arguments using the following grammar:

```
--inject <type>:<role>[:<idx>[:<step>
      [:<detect_step>[:<detect_phase>]]]]
```

where `type` $\in \{\text{format, logic, tool_call}\}$, `role` $\in \{\text{worker, planner, aggregator}\}$, `idx` is the zero-based agent index (auto-assigned if omitted), `step` is the one-based refinement step at which the fault is applied, `detect_step` is the step at which the fault is recorded in the errors array, and `detect_phase` $\in \{\text{workers, aggregator, none}\}$ controls whether the error is visible to worker sessions, to the aggregator, or to neither. As has been stated (section II-D), the error classification types here were defined based on errors observed in other existing benchmarks so that they would represent realistic error scenarios.

This separation of injection point from detection point is a key feature of AES. It enables experiments that test deferred error visibility (lag-1, lag-2, lag-3 detection), silent propagation (`detect_phase: none`), and aggregator-phase disclosure, all with the same underlying fault but different information available to downstream agents. Multiple faults can be specified by repeating `--inject`.

Format faults. The proxy replaces the model’s structured JSON chat response with malformed plain text. The agent framework receives an unparseable response for that turn, simulating output schema violations such as a worker returning a narrative explanation instead of a structured result.

Logic faults. The proxy intercepts a structurally valid JSON response and overwrites the numerical `integral` field with a scaled value (e.g., `scale = -1.0` negates the result). The response remains syntactically correct; the error is purely numerical. This simulates a plausible but incorrect computation that passes schema validation.

Tool call faults. The proxy returns an HTTP error response when the worker invokes the `compute_riemann` tool, simulating an external service failure. The agent receives a tool error rather than a numerical result for that invocation.

Synthetic response content. For worker format faults, the synthesized turn-1 response reads: “I computed the integral over $[a, b]$ on step k and it came out to roughly 2.5 or so”. This is plausible in register but structurally invalid for downstream JSON parsing. For planner format faults, the replacement describes the interval partition in prose; for aggregator format faults, it gives an approximate total in natural language. These texts simulate realistic failure modes (i.e., matching what a poorly prompted or context-confused model might actually

produce) rather than obviously malformed outputs such as empty strings or syntax errors.

For logic faults, the synthesized response is a complete, structurally valid JSON object in which the `integral` field is set to $v_{\text{correct}} \times s$, where $s = -1.0$ by default (`scale_factor` is configurable per injection spec). The `converged` flag is forced to `false` regardless of the true convergence status, preventing early termination that would reduce the fault’s opportunity to propagate into subsequent steps. For aggregator logic faults, the synthetic report sets `total_integral` = $\Sigma_w \times s$, where Σ_w is the coordinator’s own running sum of worker final values, preserving the response structure while inverting the total.

The genuine model call is always executed before fault replacement: the coordinator issues the real HTTP request to Ollama (recording it in the proxy audit trail) and then substitutes the synthetic response at the application layer. This ensures the provenance trace captures the model’s actual response to the prompt, while the workflow propagates the injected value.

Injection and detection records. In order to support post-hoc analysis, we need to have a way to track statistics about injection and error occurrences. To accommodate this, every fault produces two parallel records in the JSON output: injections and errors. The `injections[]` array receives an entry for every fault that fires, recording `error_type`, `inject_step`, `detect_step`, `detect_phase`, `detected` (bool), and `detected_at_step`. The `errors[]` array receives entries only when detection fires: immediately at the injection step (default), at the configured deferred step, or at the aggregator phase via `final_report["aggregator_detected_errors"]`. Silent faults (`detect_phase: none`) appear in `injections[]` but never in `errors[]`, so the full injection history is available for post-hoc analysis regardless of whether the fault was ever surfaced to the model.

E. Test Suite Organization

The benchmark comprises ten test groups (Table I), totaling 194 jobs across four models. Groups vary injection count, error type, step depth, worker count, context window size, compaction policy, and detection timing, providing controlled isolation of each experimental axis.

Groups A and B establish the clean-condition baseline and single-fault taxonomy. Group A records per-model ϵ under no injection, providing the reference against which all subsequent results are normalized. Group B crosses each of the three error types with each of the three agent roles (nine combinations, minus the two not evaluated for all models), holding worker count (3) and step depth (2) fixed. Each B scenario isolates exactly one axis of the fault taxonomy.

Group C tests concurrent fault compositions. Four scenarios introduce multiple simultaneous faults: two error types on two different workers (format + logic at step 1); all three types on three workers simultaneously; a cross-agent combination (planner format plus worker logic); and a staggered schedule

Group	Evaluation Axis	Steps/W	Jobs
A	Baseline (no injection)	2 / 3	4
B	Single error per type $\times$ role	2 / 3	24
C	Simultaneous multi-agent errors	2–3 / 3	16
D1	Scale — minimal	1 / 2	4
D2	Scale — medium (1 injection)	2 / 5	4
D3	Scale — large (2 injections)	2 / 8	2
E	Deep sessions (6 steps)	6 / 3–5	20
F	Context window stress	4–6 / 3–5	28
G	Detection timing variants	3–6 / 3	80
H	Forced compaction (max-turns=1)	6 / 3	12
Total			194

TABLE I: AES Test Group Summary

in which worker 0 fails at step 1, worker 1 at step 2, and worker 2 at step 3. These scenarios test whether fault effects are independent and whether multi-fault interactions produce outcomes not predictable from single-fault results.

Groups D1, D2, and D3 vary the worker count from 2 to 8. Group D1 provides a minimal clean baseline (2 workers, 1 step). Group D2 injects one `tool_call` fault at 5 workers and 2 steps. Group D3 injects two faults (logic + format) at 8 workers and 2 steps with Gemma4 only. The aggregator’s input grows linearly with worker count; these groups isolate whether aggregator arithmetic degrades as the annotated result set expands.

Group E extends the refinement depth to 6 steps across five scenarios: a clean baseline, format and `tool_call` faults at mid-session steps (3 and 4), a logic fault at step 5, and a two-fault multi-worker scenario at 5 workers with errors at steps 3 and 5. The deeper step sequence tests whether the additional correct-step context before and after a fault stabilizes or destabilizes recovery relative to 2-step sessions.

Group F constrains the Ollama context window to 2048 or 512 tokens. At 2048 tokens with 4-step sessions, context exhaustion occurs near the final step; at 512 tokens with 6-step sessions, exhaustion occurs around step 3. Seven scenarios cross-combine context limit with fault type, testing whether token truncation introduces failure modes beyond those observed under unlimited context.

Group G systematically isolates the injection/detection separation across five subgroups. G1 tests single-step detection lag (inject at step 1, report at step 2). G2 tests lags of 2 to 3 steps, including a scenario where the configured `detect_step` exceeds the run length and fires at run-end. G3 tests aggregator-phase detection, in which workers remain silent and the coordinator logs the detection in `final_report` after the aggregator completes. G4 tests silent propagation (`detect_phase: none`), where faults reach the final answer without ever appearing in any error list. G5 provides pairwise comparisons in which only the detection mode differs, directly isolating whether the information available to downstream agents via `errors[]` affects numerical outcomes.

Group H forces per-step context compaction using `--max-turns 1`: after each refinement step, the worker’s

message history is trimmed to retain only the system prompt and the four messages from the most recent step, eliminating all prior-step context. Three scenarios (clean, format fault at step 2, logic fault at step 3) test whether this hard compaction affects recovery behavior relative to the unconstrained 6-step equivalents in Group E.

IV. EVALUATIONS

A. Implementation Details

The benchmark is assessed via a collection of jobs submitted with the Slurm scheduler. Benchmark models are served locally via Ollama, which is launched per job, polled until ready, and terminated on job exit. Each job occupies an exclusive Ollama port (base 11434 + slot $\times$ 500) to prevent collisions in concurrent runs. A lightweight Flask proxy intercepts every agent-to-model HTTP POST by routing API calls. The proxy permits interaction trace recording and fault injections. The full benchmark is available on GitHub at <https://github.com/Keith-Bateman/agent-error-simulator>.

B. Testbed

Tests were performed on the 152-node Delta AI supercomputer [25] at NCSA, each node has 4x NVIDIA GH200 superchips (H100 GPU + Grace ARM CPU). The CPUs have 120GB of LPDDR5X memory and the GPUs have 96GB of HBM3. Delta AI provides access to two Luster filesystems which provide 1 TB HDD and 500 GB NVMe storage, as well as 3.5 TB local NVMe drive on each node. It utilizes a 200 Gb/s HPE SlingShot network.

C. Experimental Setup

We evaluate four open-weight models: Qwen2.5-Coder 32B (`qwen2.5-coder:32b`), Gemma4 27B (`gemma4:27b`), Mistral 7B (`mistral:latest`), and Granite4 2B (`granite4:latest`). Qwen was evaluated in an initial 50-job run. The remaining three models were evaluated jointly in a 144-job run, yielding 184 completed results (10 job failures, detailed below). All runs used a single NVIDIA GH200 GPU (60 GB HBM) per job.

D. Baseline Performance

Table II shows absolute error under clean conditions (Group A, 3 workers, 2 steps). Gemma4 achieves exact aggregation ($\epsilon = 0.0$). Qwen and Granite4 exhibit minor rounding artifacts. Mistral’s aggregator systematically under-reports the total integral by $\approx \frac{1}{3}$, yielding $\epsilon = 0.333$ despite correct worker values. This structural deficit (which occurs across every clean Mistral run in the suite) dominates Mistral’s results throughout all experiments.

E. Worker Error Recovery

Format and tool_call faults recover universally. As shown in Table III, across all four models, all step depths, context window sizes, and compaction settings, format and tool_call faults injected at non-final refinement steps produce no change in ϵ beyond the model’s baseline. At the next refinement step,

Model	Params	ϵ	Note
Gemma4	27B	0.0000000	exact aggregation
Granite4	2B	0.0008750	rounding artifact
Qwen2.5-Coder	32B	0.0003125	copies tool result exactly
Mistral	7B	0.3334375	chronic $-1/3$ under-report

TABLE II: Baseline Absolute Error (Group A, 3 Workers, 2 Steps)

Model	Format	Tool_call	Logic (non-fin)	Logic (fin)
Qwen 32B	✓	✓	✓	×
Gemma4 27B	✓	✓	✓	×
Mistral 7B	✓	✓	×	×
Granite4 2B	✓	✓	×	×

TABLE III: Worker Error Recovery Capability by Model and Error Type

the worker independently invokes `compute_riemann` and copies the correct result, overwriting the faulty turn in context history. This behavior is stable under 512-token and 2048-token context limits and under per-turn compaction (`max-turns=1`).

Logic fault recovery is model-size-dependent. As shown in Table III, Qwen (32B) and Gemma4 (27B) recover from logic faults injected at non-final steps across every tested scenario. Both models re-invoke the `compute_riemann` tool at the subsequent step and use the correct tool output, ignoring the prior step’s injected wrong value. Mistral (7B) and Granite4 (2B) do not recover: these models read the prior step’s injected value from context history and reproduce it as their own output at the next step, without the `injected=true` annotation. The error is now self-generated, structurally valid, and indistinguishable from a model-produced mistake.

The recovery threshold appears to lie between 7B and 27B parameters and is robust to detection mode, context configuration, and step depth. When a logic fault is injected at the final refinement step, even capable models (Qwen, Gemma4) can propagate the wrong value, since no subsequent step can correct it. This is the only reliable propagation path for worker logic faults in larger models.

F. Role-Level Impact

The severity of a fault depends strongly on which agent role is targeted (Fig. 2).

Planner faults are harmless in all tested configurations. Format faults on the planner are absorbed by the framework’s fallback parsing; the `injected_errors` array remains empty and the workflow proceeds normally. The planner produces the initial task decomposition; errors in this role are buffered before any computation begins.

Worker faults propagate under the conditions described in Section IVC. Cross-agent injection (simultaneous planner + worker faults) yields the same worker-only outcome, confirming that planner fault absorption is independent of worker behavior.

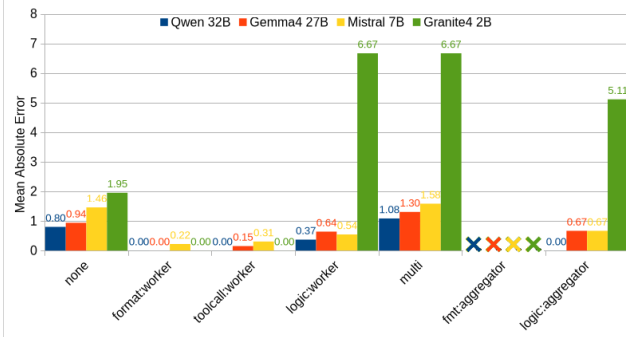


Fig. 2: Absolute error by fault type and agent role. Planner faults are uniformly harmless, and therefore not shown (error was always 0). Worker logic faults propagate for Mistral and Granite4. Aggregator logic faults are catastrophic for all models.

Model	Workers	Σ Finals	Agg Total	ϵ
Qwen 32B	5	5.9999	6.0049	0.005
Qwen 32B	8	5.9999	5.9094	0.091
Gemma4 27B	5	5.9999	6.9077	0.908
Gemma4 27B	8	5.9999	6.1904	0.190
Mistral 7B	5	5.9999	6.1928	0.193

TABLE IV: Aggregator Arithmetic Error at Scale (Workers Correct, $\Sigma \approx 6.0$)

Aggregator faults are catastrophic for all four models. A format fault on the aggregator causes it to return no valid JSON, producing a NULL result in every case (not shown as Mean Absolute Error in the graph, but an error nonetheless). A logic fault on the aggregator (scale = -1.0) causes it to report the negated total, yielding $\epsilon \approx 12.0$ ($\approx \times 38,000$ above Qwen baseline, identical in sign and magnitude across all models). Because no agent processes the aggregator’s output, there is no recovery path regardless of model size.

G. Scale Effects

When the workflow scales to 5 or 8 workers with error annotations present in the aggregator’s context, aggregator arithmetic accuracy reaches a degraded state for all models (Table IV). Workers consistently sum to the correct value ($\Sigma \approx 6.0$); the aggregator introduces error when parsing an annotated result set. While Gemma4 seems to improve with 8 workers, it is unclear whether this trend would continue at higher worker counts.

The variable degradation of Gemma4 at these scales (the 5-worker error of $+0.91$ exceeding the 8-worker error of $+0.19$) suggests that aggregation accuracy depends on context layout rather than worker count alone. Granite4 produced no results for 5-worker configurations (Groups D2, H), likely due to timeout or model instability, preventing scale analysis for this model.

Detection Mode	ϵ
Immediate (workers phase)	0.0000781
Lag-1 step	0.0000781
Lag-2 steps	0.0000781
Lag-3 steps	0.0000781
Aggregator phase only	0.0000781
Silent (detect_phase: none)	0.0000781
Beyond run end	0.0000781

TABLE V: Detection Mode Does Not Affect Absolute Error (Qwen 32B, Logic Worker Fault)

H. Context Window and Compaction

Constraining the context window to 512 or 2048 tokens (Group F) and forcing per-turn compaction (Group H, max_turns=1) have no effect on format or tool_call fault recovery for any model (see comprehensive figure 3 for details). Logic fault behavior is similarly unchanged: models that recovered in full-context runs continue to recover, and models that did not continue to propagate.

A notable exception is Gemma4 under a 512-token context limit with no injected faults (F_ctx512_clean): the aggregator returned 4.0 instead of 6.0 ($\epsilon = 2.0$). The same 4.0 aggregator output appears in two G-group scenarios where error annotations are loaded into the aggregator’s context without a context constraint. This suggests that Gemma4’s aggregator conflates context incompleteness with a partial worker result set (whether through token truncation or annotation-induced confusion) and sums only a subset of workers. This failure mode is distinct from the arithmetic degradation seen at scale and does not affect other models.

I. Detection Timing

All G-group experiments confirm that detection timing has no causal effect on the final result (see table V). Whether a fault is flagged immediately (detect_phase: workers), deferred by one to three steps, disclosed only at the aggregator phase, entirely silent (detect_phase: none), or specified beyond the run’s end, the numerical output is unchanged. The fault annotation is recorded in the audit trail at the specified detection point, but its presence or timing does not alter model behavior during execution.

This result holds across all four models and all tested detection variants. The final output is determined solely by whether the worker recovered the correct tool result and whether the aggregator summed faithfully, not by the contents of the errors array.

J. Cross-Model Comparison and Summary

Fig. 3 summarizes absolute error across the test suite by model. The most consequential finding is the logic recovery threshold between 7B and 27B parameters, which divides the four models into two qualitatively distinct behavioral classes.

Models above the threshold (Qwen 32B, Gemma4 27B) invoke the `compute_riemann` tool independently at each refinement step and use the correct tool output, discarding

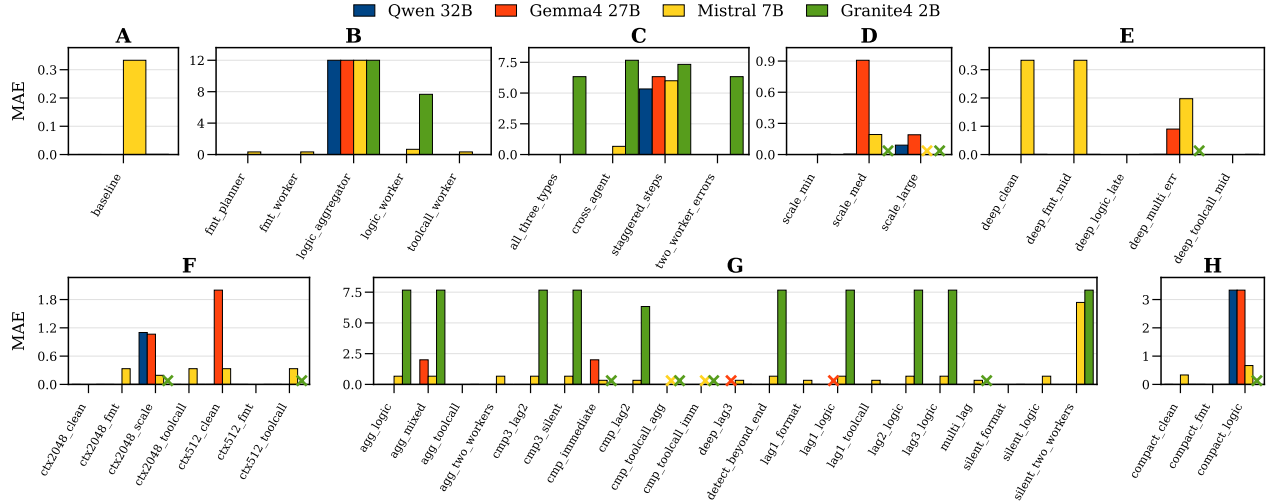


Fig. 3: Absolute error across test scenarios by model. The Gemma4 and Qwen baselines are near zero. The Mistral chronic offset ($\epsilon \approx 0.333$) is visible across most Mistral results. Catastrophic aggregator-level injections reach $\epsilon \approx 12.0$ for all models.

prior history values. Models below the threshold (Mistral 7B, Granite4 2B) treat the prior step’s numerical result as ground truth and propagate it, causing logic errors to persist across all subsequent steps regardless of injection timing.

Mistral’s chronic $-1/3$ aggregator under-reporting creates a problem in some scenarios: unrecovered logic worker errors can partially cancel the aggregator’s baseline offset, producing near-zero ϵ despite both a propagated worker error and the sum mistakes of the aggregator. This coincidental compensation is not robustness (the underlying errors remain) and complicates interpretation of Mistral results without accounting for the baseline offset.

Granite4’s instability under multi-worker and extended-step configurations (8 of 10 job failures concentrated in Groups D2, E, F, G, and H) limits its use for scale or deep-session experiments. Gemma4’s context-truncation aggregator failure (4.0 output under 512-token limits) provides a useful sentinel condition for detecting context loss in aggregation pipelines.

V. CONCLUSIONS

We presented the Agent Error Simulator (AES), a benchmark for evaluating fault tolerance in multi-agent LLM workflows through controlled, declarative fault injection. Across 194 jobs and four models, we find that fault impact is determined primarily by three factors: the type of fault (logic faults are the most dangerous, propagating through pattern-following in smaller models), the agent role targeted (aggregator injection is universally catastrophic), and the model’s parameter scale (logic recovery requires at least 27B parameters in our evaluation set). Detection timing is irrelevant to outcome; context window constraints and compaction do not affect recovery of non-logic faults.

These findings have practical implications for multi-agent system design. Late multi-input stages such as the aggregator

represent critical points that should be hardened through validation or redundancy. Workflows requiring reliable error recovery should use models of sufficient scale, and results from small models should be interpreted with an understanding of their error-propagation behavior. AES provides a reproducible foundation for evaluating these properties as new models and agent architectures emerge.

Future work will extend AES to more complex task types, repeated trial variance estimation, additional analysis phases, broader model families, and automated detection of the conditions under which agents self-correct.

ACKNOWLEDGMENTS

This material is based upon work supported in part by the National Science Foundation (NSF), Division of Computer and Network Systems (CISE/CNS), under Grants CCRI-CISE 2346504, CSSI-2411386, CSSI-2411318, and the Department of Energy (DOE), Office of Advance Scientific Computing Research (ASCR), under Grants DE-AC02-06CH11357 and DE-SC0024593. This research used the DeltaAI system at the National Center for Supercomputing Applications through allocation CIS250329 from the ACCESS program, supported by NSF award OAC 2320345. Claude was used to assist in initial drafting of this paper for sections I, IIA-C, IIIC-E (excluding figure 1), IVA, IVC-J (excluding figures), and V. Ideas present are our own and final drafts were subject to review and editing by the authors.

REFERENCES

- [1] Anonymous, “Scifi: A safe, lightweight, user-friendly, and fully autonomous agentic ai workflow for scientific applications,” *arXiv*, vol. abs/2604.13180, 2026.
- [2] Anonymous, “A cloud-based multi-agentic workflow for science,” *arXiv*, vol. abs/2601.12607, 2026.

- [3] Z. S. Siegel, S. Kapoor, N. Nagdir, B. Stroebel, and A. Narayanan, "Core-bench: Fostering the credibility of published research through a computational reproducibility agent benchmark," in *Thirty-eighth Conference on Neural Information Processing Systems (NeurIPS)*, 2024.
- [4] Q. Luo, L. Xu, Q. Lin, S. Wu, R. c. Mao, C. Wang, H. Feng, B. Huang, and Z. Du, "Geojson agents: a multi-agent llm architecture for geospatial analysis-function calling vs code generation," *arXiv*, 2025.
- [5] J. Ock, R. S. Meda, S. Badrinarayanan, N. S. Aluru, A. Chandrasekhar, and A. B. Farimani, "Large language model agent for modular task execution in drug discovery," *arXiv*, 2025.
- [6] M. Tran, H. Nguyen, S. Smith, and A. Patel, "The next paradigm in bioinformatics: a review of multi-agent systems and foundational models for end-to-end scientific discovery," *Briefings in Bioinformatics*, vol. 27, no. 3, p. bbag245, 2026.
- [7] Z. Liu, Y. Wang, R. Zhang, S. Huang, and X. Wu, "Understanding the information propagation effects of communication topologies in llm-based multi-agent systems," in *Findings of the Association for Computational Linguistics: EMNLP 2025*, pp. 7231–7245, Association for Computational Linguistics, 2025.
- [8] Anonymous, "Metacognitive self-correction for multi-agent system via prototype-guided next-execution reconstruction," in *International Conference on Learning Representations (ICLR)*, 2026.
- [9] X. Liu, H. Yu, H. Zhang, Y. Xu, X. Lei, H. Lai, Y. Gu, H. Ding, K. Men, K. Yang, *et al.*, "Agentbench: Evaluating llms as agents," in *International Conference on Learning Representations*, vol. 2024, pp. 52989–53046, 2024.
- [10] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan, "Swe-bench: Can language models resolve real-world github issues?," in *International Conference on Learning Representations*, vol. 2024, pp. 54107–54157, 2024.
- [11] D. Zhang, S. Zhoubian, M. Cai, F. Li, L. Yang, W. Wang, T. Dong, Z. Hu, J. Tang, and Y. Yue, "Datascibench: An llm agent benchmark for data science," *arXiv*, vol. abs/2502.13897, 2025.
- [12] Anonymous, "Dabstep: Data agent benchmark for multi-step reasoning," in *OpenReview*, 2025.
- [13] S. Tan, "Medresearchbench: A multi-domain benchmark for evaluating ai research agents on clinical medical research," *medRxiv*, 2026.
- [14] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "React: Synergizing reasoning and acting in language models," *arXiv preprint arXiv:2210.03629*, 2022.
- [15] Q. Wu, G. Bansal, J. Zhang, Y. Wu, B. Li, E. Zhu, L. Jiang, X. Zhang, S. Zhang, J. Liu, *et al.*, "Autogen: Enabling next-gen llm applications via multi-agent conversations," in *First conference on language modeling*, 2024.
- [16] M. Cemri, M. Z. Pan, S. Yang, L. A. Agrawal, B. Chopra, R. Tiwari, K. Keutzer, A. Parameswaran, D. Klein, K. Ramchandran, *et al.*, "Why do multi-agent llm systems fail?," *Advances in Neural Information Processing Systems*, vol. 38, 2026.
- [17] Z. Chen, S. Chen, Y. Ning, Q. Zhang, B. Wang, B. Yu, Y. Li, Z. Liao, C. Wei, Z. Lu, *et al.*, "Scienceagentbench: Toward rigorous assessment of language agents for data-driven scientific discovery," in *International Conference on Learning Representations*, vol. 2025, pp. 96934–96990, 2025.
- [18] E. Lai, G. Vitagliano, Z. Zhang, O. Chabra, S. Sudhir, A. Zeng, A. A. Zabreyko, C. Li, F. Kossmann, J. Ding, *et al.*, "Kramabench: A benchmark for ai systems on data-to-insight pipelines over data lakes," *arXiv preprint arXiv:2506.06541*, 2025.
- [19] Y. Qi, H. Peng, X. Wang, A. Xin, Y. Liu, B. Xu, L. Hou, and J. Li, "Agentif: Benchmarking instruction following of large language models in agentic scenarios," *arXiv preprint arXiv:2505.16944*, 2025.
- [20] D. Nichols, J. H. Davis, Z. Xie, A. Rajaram, and A. Bhatele, "Can large language models write parallel code?," in *Proceedings of the 33rd International Symposium on High-Performance Parallel and Distributed Computing*, pp. 281–294, 2024.
- [21] A. Basiri, N. Behnam, R. De Rooij, L. Hochstein, L. Kosewski, J. Reynolds, and C. Rosenthal, "Chaos engineering," *IEEE software*, vol. 33, no. 3, pp. 35–41, 2016.
- [22] E. Y. Chang and L. Geng, "Sagallm: context management, validation, and transaction guarantees for multi-agent llm planning," *arXiv preprint arXiv:2503.11951*, 2025.
- [23] D. Nichols, J. H. Davis, Z. Xie, A. Rajaram, and A. Bhatele, "Can large language models write parallel code?," in *Proceedings of the 33rd International Symposium on High-Performance Parallel and Distributed Computing*, HPDC '24, (New York, NY, USA), Association for Computing Machinery, 2024.
- [24] Y. Huang, S. Di, and G. Li, "Not all errors are equal: A systematic study of error propagation in large language model inference," *arXiv preprint arXiv:2606.02430*, 2026.
- [25] National Center for Supercomputing Applications, "DeltaAI advanced computing and data resource." <https://delta.ncsa.illinois.edu/>, 2024. Supported by NSF award OAC-2320345.